

Answer Key: The Smart Dice Mission

Phase 1: The Basic Dice (Review and Launch)

Concept Check Question	Expected Answer/Explanation
Input: What must you do to the micro:bit's code for it to start running?	The code must be flashed (or downloaded) from the computer to the micro:bit device.
Output: What kind of output does this initial code produce? What specific hardware component is responsible for showing the number?	It produces a visual output (the number). The component is the LED display (the 5x5 grid of lights).
Core Components: What does the import random line allow the code to do?	It imports the random module , which contains tools (like randint) to generate random numbers.
Core Components: What is the role of the while True: loop in this code?	It creates an infinite loop , ensuring the code inside it repeats forever. This makes the dice keep rolling automatically.

Phase 2: Activating the Dice with a Gesture (Input Sensor Challenge)

Goal: Modify the code so the dice only rolls a new number when the micro:bit is shaken.

Conceptual Clue	Expected Solution/Concept
New Input: Which part of the micro:bit is responsible for sensing motion?	The accelerometer sensor.
Detection: How do you check if a specific gesture (like 'shake') has just occurred?	Use the accelerometer.was_gesture() function.
Control Structure: Where should the core rolling code go?	Inside an if statement that checks for the shake gesture.

Expected Code Logic: The code moves from a continuous loop (while True: with sleep(2000)) to a gesture-activated loop:

```
# The loop is now waiting for a condition to be met
while True:
    if accelerometer.was_gesture('shake'):
        # Only run these lines if a shake happened:
        x = random.randint(1,6)
        display.show(x)
        # No need for sleep() here, as it waits for the next shake
```

Phase 3: Celebrate the Roll (Audio Output Challenge)

Goal: Make the `micro:bit` play a short sound/jingle immediately before the new dice number is displayed.

Conceptual Clue	Expected Solution/Concept
New Tool: What is the name of the module that handles sound and music?	The music module.
Playing a Tune: What is the specific command to play a sequence of notes?	<code>music.play()</code>
Speaker: What hardware component is used to output the sound?	Either the on-board speaker (on newer <code>micro:bit</code> models) or connecting headphones/piezo buzzer to Pin 0 and GND.

Expected Code Logic Addition: The student must add `import music` at the top and insert a `music.play()` command *before* the roll:

```
import music # ADDED

# ... inside the while True loop ...
if accelerometer.was_gesture('shake'):
    music.play(['c4:1', 'e4:1', 'g4:2'], wait=True) # ADDED/Example tune
    x = random.randint(1,6)
    display.show(x)
```

Phase 4: Speak the Score (Conditional Audio Output Challenge)

Goal: Make the `micro:bit` say the number it rolled after it is displayed, with different announcements.

Conceptual Clue	Expected Solution/Concept
Another New Tool: What is the name of the module that handles speech synthesis ?	The speech module.
Spoken Output: What is the command that allows you to make the <code>micro:bit</code> speak?	<code>speech.say()</code>
Making Decisions: How do you say a <i>different</i> phrase for each number?	Use a sequence of if and elif (else if) statements to check the value of <code>x</code> .

Expected Code Logic Addition: The student must add `import speech` at the top and add the conditional logic after `display.show(x)`:

```

import speech # ADDED

# ... inside the while True loop ...
if accelerometer.was_gesture('shake'):
    # ... music and roll code here ...
    display.show(x)

    if x == 6:          # ADDED
        speech.say('six, six!')
    elif x == 5:        # ADDED
        speech.say('five')
    # ... continue with elif for 4, 3, 2, 1 ...

```

Phase 5: The Button Backup (Alternative Input Challenge)

Goal: Allow the user to roll the dice by pressing either Button A or Button B, in addition to shaking it.

Conceptual Clue	Expected Solution/Concept
Alternative Input: Which part of the micro:bit is responsible for detecting a button press ?	The dedicated Button A and Button B components.
Detecting a Press: Find the command that checks if a specific button has been pressed .	button_a.was_pressed() or button_b.was_pressed().
Combining Conditions: How do you check for EITHER the shake OR a button press?	Use the logical operator or within the if statement.

Expected Code Logic Addition: The student must combine all three conditions using or:

```

# Final Combined Logic
while True:
    if accelerometer.was_gesture('shake') or button_a.was_pressed() or button_b.was_pressed():

        # This entire block runs if ANY of the conditions above are TRUE
        music.play(['c4:1', 'e4:1', 'g4:2'], wait=True)
        x = random.randint(1,6)
        display.show(x)

        if x == 6:
            speech.say('six, six!')
        # ... elif statements continue ...

```

```
# Imports go at the top
```

```
from microbit import *
```

```
import random
```

```
# step one
```

```
while True:
```

```
    x = random.randint(1,6)
```

```
    display.show(x)
```

```
    sleep(2000)
```

```
# 2: shake
```

```
# 3: add music before
```

```
import music
```

```
# 4: speak it out
```

```
import speech
```

```
while True:
```

```
    if accelerometer.was_gesture('shake'):
```

```
        music.play(['g4:2', 'a4:2', 'b4:2', 'g4:2', 'c5:6'], wait=True)
```

```
x = random.randint(1,6)

display.show(x)

if x == 6:

    speech.say('six, seven?')

elif x == 5:

    speech.say('fi i i i ive')

elif x == 4:

    speech.say('for or or or')

elif x == 3:

    speech.say('threeeeeee')

elif x == 2:

    speech.say('twooooo')

elif x == 1:

    speech.say('it is onee!')
```

#5 click button will trigger as well